ENTERPRISE NETWORK PROJECT
Ubuntu 22.04 LTS — Full Infrastructure Guide
Corrected & Verified Edition

Domain:	enterprise.local
Subnet:	192.168.100.0/24
Platform:	VirtualBox + Ubuntu 22.04 LTS Server
VMs:	10 (Router, DNS, File, Web, SMTP, Monitor, Backup, 2×Client)
Revised:	May 2026

Team Members

ABRAHAM	Router / DHCP / Firewall + SMTP Server
ROBINE	DNS Server
MANDA	File Server (Samba)
ABBA	Web Server (Apache)
KANA	Monitoring (Zabbix)
JOYCE	Backup Server + Client VMs

Confidential — For team use only

Introduction & Project Overview

This document is the sole reference your team will use throughout the project. It is written to be executed literally, command by command, with zero ambiguity. By following it in order, your team will build a fully functional, production-like enterprise network running entirely inside VirtualBox virtual machines on Ubuntu 22.04 LTS Server.

The network replicates what you would find in a real small-to-medium enterprise: a router with NAT and firewall rules, a DHCP server assigning addresses automatically, an authoritative DNS server for the private domain `enterprise.local`, a Samba file server, an Apache web server, an SMTP mail server (Postfix + Dovecot), a Zabbix monitoring server, a nightly rsync backup server, and two client workstations.

The original guide configured **Adapter 1 as NAT** and **Adapter 2 as Internal Network**. With this setup, VMs on the same physical host can reach each other, but the router VM has no way to carry traffic *out* to the real LAN or internet on behalf of the internal network, and other physical machines cannot reach your VMs at all.

The fix applied throughout this document:

- **Adapter 1 → Bridged Adapter** (attached to your real NIC). This gives the router VM a real IP on your local network so it can communicate with the outside world and be reachable from other hosts.
- **Adapter 2 → Internal Network** (`intnet_enterprise`). This is the private LAN used by all VMs.
- **Adapter 3 → Host-Only Adapter** (for SSH from host).

On the router VM, Adapter 1 (Bridged) faces the real network. The iptables MASQUERADE rule is applied on this bridged interface. All other VMs use only Adapter 2 (Internal Network) and Adapter 3 (Host-Only).

What Makes This Project Stand Out

- **Full service integration:** every service is configured to talk to every other — DHCP provides DNS, DNS resolves all hostnames including smtp, the SMTP server accepts mail for enterprise.local, Zabbix monitors all VMs via agent, and backup pulls all critical configs nightly.
- **Live intranet dashboard:** the web server hosts a PHP page that probes all service IPs (including the SMTP VM) and shows UP/DOWN status in real time.
- **SMTP mail infrastructure:** Postfix (MTA) + Dovecot (IMAP) provide enterprise email for the enterprise.local domain, with mailboxes for all team members.
- **Nightly automated backup:** rsync over SSH pulls every critical config directory; old backups are pruned after 7 days.
- **Monitoring with alerts:** Zabbix tracks CPU, memory, disk, and process availability for all hosts including the SMTP VM.
- **One-command test suite:** Section 12 provides a systematic, numbered test for every service.

Architecture Summary

All VMs reside on the VirtualBox internal network `intnet_enterprise` (subnet `192.168.100.0/24`). The router VM bridges this internal network to the outside world via its **Bridged Adapter** on Adapter 1. Static IPs are assigned to all servers; client machines receive dynamic IPs from the DHCP server on router-vm.

Component	VM Name	IP / Range	Key Services
Internet (Bridged)	—	enp0s3 (DHCP from real LAN)	External access for all VMs
Router / GW	router-vm	192.168.100.1	iptables NAT, DHCP, IP forward
DNS Server	dns-vm	192.168.100.10	Bind9 authoritative + recursive
File Server	fileserver-vm	192.168.100.11	Samba — public + private shares
Web Server	webserver-vm	192.168.100.14	Apache + PHP dashboard
SMTP Server	smtp-vm	192.168.100.15	Postfix (MTA) + Dovecot (IMAP
Monitoring	monitoring-vm	192.168.100.12	Zabbix 6.4 + agents on all VMs
Backup Server	backup-vm	192.168.100.13	rsync over SSH, nightly cron
Client 1	client1-vm	192.168.100.50–200	DHCP, test workstation
Client 2	client2-vm	192.168.100.50–200	DHCP, test workstation

Prerequisites — Every Team Member

Before you clone or start any VM, verify the following on your host machine:

- VirtualBox **6.1 or later** installed on your host OS.
- Base OVA file *`ubuntu-base.ova`* obtained from the team leader.
- Host resources: at least **4 GB free RAM** (2 GB per running VM) and **40 GB free disk space**.
- CPU virtualisation enabled in BIOS/UEFI (Intel VT-x or AMD-V).
- A working **physical NIC** on the host that is connected to your local network (required for the Bridged Adapter on the router VM to get a real IP).

Every team member must follow Section 4 (Cloning) independently and completely before attempting their own service configuration. Do not skip the post-clone initialisation — duplicate machine IDs and SSH host keys cause hidden, hard-to-debug problems.

IP Address Plan & Hostname Table

The entire internal network is `192.168.100.0/24`. All servers use static IPs; client machines receive IPs dynamically from the DHCP server on router-vm. Every team member must know the IPs of all VMs.

VM Assignment Table

Hostname	Role	Static IP	DHCP Range	Member
router-vm	Router / DHCP / Firewall	192.168.100.1	N/A	ABRAHAM
dns-vm	DNS Server (Bind9)	192.168.100.10	N/A	ROBINE
fileserver-vm	Samba File Server	192.168.100.11	N/A	MANDA
webserver-vm	Apache + PHP Web Server	192.168.100.14	N/A	ABBA
smtp-vm	SMTP/IMAP (Postfix+Dovecot)	192.168.100.15	N/A	ABRAHAM
monitoring-vm	Zabbix Monitoring Server	192.168.100.12	N/A	KANA
backup-vm	Backup Server (rsync)	192.168.100.13	N/A	JOYCE
client1-vm	Test Workstation 1	Dynamic	.50—.200	JOYCE
client2-vm	Test Workstation 2	Dynamic	.50—.200	JOYCE

DNS Name Table — enterprise.local

DNS Name	Resolves To	Notes
router.enterprise.local	192.168.100.1	Gateway / DHCP
dns.enterprise.local	192.168.100.10	Authoritative DNS
files.enterprise.local	192.168.100.11	Samba shares
webserver.enterprise.local	192.168.100.14	Apache dashboard
www.enterprise.local	192.168.100.14	CNAME → webserver
smtp.enterprise.local	192.168.100.15	SMTP/IMAP mail server
mail.enterprise.local	192.168.100.15	CNAME → smtp
monitor.enterprise.local	192.168.100.12	Zabbix UI
backup.enterprise.local	192.168.100.13	Backup server

Cloning the Base VM — Every Team Member

The team leader has prepared *`ubuntu-base-template.ova`* with Ubuntu 22.04 LTS, user `sysadmin` (password: `admin123`), SSH server, and core tools. No services are pre-installed. Each person imports this OVA once and performs all steps below.

Import the OVA & Configure Network Adapters

1. Go to **File** → **Import Appliance**, select *`ubuntu-base.ova`*.
2. Set the VM Name to your assigned name from the IP table.
3. Under **MAC Address Policy**, choose *"Generate new MAC addresses for all network adapters"* — duplicate MACs cause total network failure.
4. Click **Import** and wait.

Before starting the VM, open **Settings** → **Network** and configure adapters **exactly** as follows:

Adapter	Attached To	Network Name / NIC	Prom. Mode	Purpose
1	**Bridged Adapter**	Your physical NIC (e.g. `eth0`/`Wi-Fi`)	Allow All	Real LAN / Interr
2	Internal Network	`intnet_enterprise`	Allow All	LAN — main proj
3	Host-only Adapter	VirtualBox Host-Only	Deny	SSH from host

Why Bridged instead of NAT?

With *NAT*, VirtualBox creates a private network visible only *within* the same physical host. The router VM's external interface (`enp0s3`) receives a VirtualBox-internal IP (e.g. 10.0.2.15) that no other physical machine can route to, and the MASQUERADE rule that should forward internal traffic exits on this non-routable address. As a result, VMs on different physical hosts — or even the host itself at the network level — cannot initiate connections to your internal network.

With *Bridged Adapter*, the router VM's `enp0s3` gets a real IP on your local network (e.g. 192.168.1.x). The MASQUERADE rule correctly exits on a reachable address, and the entire `intnet_enterprise` subnet appears accessible from any machine on your real LAN.

Non-router VMs (all except router-vm): These VMs only need **Adapter 2** (Internal Network) and **Adapter 3** (Host-Only). Adapter 1 (Bridged) is *only required on router-vm* because it is the sole gateway. Enabling a bridged adapter on every VM would bypass the router and break the network topology.

First Boot & Post-Clone Initialisation

```
# 1. Regenerate unique SSH host keys
sudo rm -f /etc/ssh/ssh_host_*
sudo ssh-keygen -A
sudo systemctl restart ssh

# 2. Generate a fresh unique machine-id
sudo rm -f /etc/machine-id /var/lib/dbus/machine-id
sudo systemd-machine-id-setup

# 3. Set hostname (replace YOUR_HOSTNAME with your assignment)
#    Examples: router-vm | dns-vm | smtp-vm | fileserver-vm
#              webserver-vm | monitoring-vm | backup-vm | client1-vm | client2-vm
sudo hostnamectl set-hostname YOUR_HOSTNAME
echo "127.0.1.1 YOUR_HOSTNAME" | sudo tee -a /etc/hosts
```

```
# 4. Verify
hostname    # Should print: YOUR_HOSTNAME
```

The original used `127.0.0.1 YOUR_HOSTNAME`. The correct entry for a non-loopback hostname on Ubuntu is `127.0.1.1 YOUR_HOSTNAME`. Using `127.0.0.1` confuses some services (Postfix, Zabbix) that test whether a hostname resolves to "myself".

Netplan Configuration

First identify your interface names:

```
ip link show
# Typical output:
#   2: enp0s3  -- Bridged (router-vm) / unused (other VMs)
#   3: enp0s8  -- Internal Network (LAN)
#   4: enp0s9  -- Host-Only (SSH)
# NOTE: Your names may differ. Always run 'ip link show'; never assume.
```

Static IP Netplan — All Servers Except Clients and Router

Create or replace */etc/netplan/01-netcfg.yaml* with the following. Substitute `X` with your VM's last octet (10, 11, 12, 13, 14, or 15):

```
# /etc/netplan/01-netcfg.yaml  ---  STATIC IP SERVERS (non-router)
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s8:                                  # Internal Network (LAN)
      dhcp4: false
      addresses:
        - 192.168.100.X/24                   # Replace X with your last octet
      routes:
        - to: default
          via: 192.168.100.1                 # router-vm is always the gateway
      nameservers:
        addresses: [192.168.100.10]          # Our DNS; use 8.8.8.8 temporarily if dns-vm not ye
    enp0s9:                                   # Host-Only (SSH from host)
      dhcp4: true
```

The original netplan for server VMs included an `enp0s3: dhcp4: true` entry. Non-router VMs do **not** have a Bridged Adapter (Adapter 1 should be *disabled* in VirtualBox for all VMs except router-vm). Configuring a missing interface in Netplan causes a boot warning and can randomly delay startup. Remove it.

Router-VM Netplan — ABRAHAM Only

The router-vm is the only VM with three active adapters:

```
# /etc/netplan/01-netcfg.yaml  ---  ROUTER-VM (ABRAHAM only)
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:                              # Bridged --- gets IP from real LAN DHCP
      dhcp4: true
    enp0s8:                              # Internal Network --- static gateway IP
      dhcp4: false
      addresses:
        - 192.168.100.1/24
      # NOTE: No default route here; the bridged adapter provides it.
    enp0s9:                              # Host-Only (SSH from host)
      dhcp4: true
```

The original added `routes: - to: default via: 192.168.100.1` on `enp0s8`. The router VM IS the gateway; setting a default route pointing to itself causes a routing loop and breaks NAT. The default route must come from `enp0s3` (Bridged) only.

Client VMs Netplan — JOYCE Only

```
# /etc/netplan/01-netcfg.yaml  ---  CLIENT VMs (JOYCE only)
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s8:                              # Receives IP from router-vm DHCP
      dhcp4: true
    enp0s9:                              # Host-Only (SSH from host)
      dhcp4: true
```

Apply Netplan on every VM after editing:

```
sudo chmod 600 /etc/netplan/01-netcfg.yaml
sudo netplan apply
sudo reboot

# After reboot verify:
ip addr show enp0s8          # Check your static/DHCP IP
ip route show default        # Should route via 192.168.100.1 (except on router-vm)
ping -c 3 192.168.100.1      # Verify router reachable (once router-vm is up)
```

ABRAHAM — Router / DHCP / Firewall (router-vm)

VM:	router-vm
Role:	Router / NAT / DHCP / Firewall (iptables)
IP:	192.168.100.1 (enp0s8) + DHCP on enp0s3 (Bridged)

The router VM is the most critical machine. It must be running before any other VM can reach the internet or get a DHCP lease. Configure it first.

Enable IP Forwarding

```
# Enable immediately (survives until reboot):
sudo sysctl -w net.ipv4.ip_forward=1

# Make permanent:
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p

# Verify:
sysctl net.ipv4.ip_forward      # Expected: net.ipv4.ip_forward = 1
```

Configure iptables NAT & Forwarding

The MASQUERADE rule must target the **interface that faces the real network**. With the original NAT adapter this was also `enp0s3`, but it was a private VirtualBox address. With the Bridged Adapter, `enp0s3` now carries a real LAN IP — so the command stays the same, but it now works correctly because traffic exits on a routable address.

```
# Write persistent iptables startup script:
sudo tee /etc/network/if-pre-up.d/iptables << 'EOF'
#!/bin/sh
# NAT: masquerade all LAN traffic going out through the bridged adapter (enp0s3)
iptables -t nat -A POSTROUTING -s 192.168.100.0/24 -o enp0s3 -j MASQUERADE
# FORWARD: allow LAN -> internet
iptables -A FORWARD -i enp0s8 -o enp0s3 -j ACCEPT
# FORWARD: allow established/related internet -> LAN
iptables -A FORWARD -i enp0s3 -o enp0s8 -m state \
    --state ESTABLISHED,RELATED -j ACCEPT
EOF
sudo chmod +x /etc/network/if-pre-up.d/iptables

# Apply rules now (without rebooting):
```

```
sudo iptables -t nat -A POSTROUTING -s 192.168.100.0/24 -o enp0s3 -j MASQUERADE
sudo iptables -A FORWARD -i enp0s8 -o enp0s3 -j ACCEPT
sudo iptables -A FORWARD -i enp0s3 -o enp0s8 -m state \
    --state ESTABLISHED,RELATED -j ACCEPT

# Install iptables-persistent to survive reboots:
sudo apt install -y iptables-persistent
# Prompt "Save current IPv4 rules?" -> YES

sudo netfilter-persistent save

# Verify:
sudo iptables -L FORWARD -v -n
sudo iptables -t nat -L POSTROUTING -v -n
```

Install & Configure ISC-DHCP Server

```
sudo apt update
sudo apt install -y isc-dhcp-server
```

Edit */etc/dhcp/dhcpd.conf* (replace entire content):

```
option domain-name "enterprise.local";
option domain-name-servers 192.168.100.10;  # Points clients to our DNS server

default-lease-time 600;
max-lease-time 7200;
ddns-update-style none;
authoritative;

subnet 192.168.100.0 netmask 255.255.255.0 {
  range 192.168.100.50 192.168.100.200;
  option routers 192.168.100.1;
  option subnet-mask 255.255.255.0;
  option domain-name-servers 192.168.100.10;
  option domain-name "enterprise.local";
}
```

Tell the DHCP server which interface to listen on:

```
sudo nano /etc/default/isc-dhcp-server
# Find:    INTERFACESv4=""
# Change:  INTERFACESv4="enp0s8"

sudo systemctl restart isc-dhcp-server
sudo systemctl enable isc-dhcp-server
sudo systemctl status isc-dhcp-server   # Look for: Active: active (running)
sudo journalctl -u isc-dhcp-server -n 30 --no-pager  # If it fails
```

Router Verification Checklist

```
sysctl net.ipv4.ip_forward                          # Must show: 1
sudo iptables -t nat -L POSTROUTING -v -n | grep MASQUERADE
sudo systemctl status isc-dhcp-server | grep Active
ping -c 3 8.8.8.8                                   # Internet from router
ip addr show enp0s3                                 # Must show a real LAN IP (not 10.0.2.x)
cat /var/lib/dhcp/dhcpd.leases                      # DHCP leases (after clients connect)
```

ABRAHAM — SMTP / Mail Server (smtp-vm)

VM:	smtp-vm
Role:	SMTP Server — Postfix (MTA) + Dovecot (IMAP)
IP:	192.168.100.15

Clone the base OVA a second time, name it `smtp-vm`, set its static IP to `192.168.100.15` on `enp0s8` (Internal Network), and follow all post-clone steps (Section 4) before continuing. **smtp-vm does not need a Bridged Adapter; it accesses the internet through the router.**

This VM provides enterprise email for the `enterprise.local` domain using:

- **Postfix** — Mail Transfer Agent (MTA) — handles sending and receiving mail on port 25 (SMTP) and 587 (submission).
- **Dovecot** — IMAP server — lets clients retrieve their mail on port 143.
- **Mailboxes** — one per team member, stored in Maildir format under */home/<user>/Maildir/*.

Install Postfix & Dovecot

```
sudo apt update
sudo apt install -y postfix postfix-utils dovecot-core dovecot-imapd \
    mailutils swaks

# During Postfix installation you will see a dialog:
#   General type of mail configuration:  --> "Internet Site"
#   System mail name:                    --> enterprise.local
#
# If you miss the prompt, reconfigure with:
#   sudo dpkg-reconfigure postfix
```

Configure Postfix

```
sudo nano /etc/postfix/main.cf
# Paste / verify these lines (add missing ones, correct existing ones):
```

```
myhostname = smtp.enterprise.local
mydomain = enterprise.local
myorigin = $mydomain
inet_interfaces = all
inet_protocols = ipv4
mydestination = $myhostname, enterprise.local, smtp, localhost.localdomain, localhost
mynetworks = 192.168.100.0/24, 127.0.0.0/8
home_mailbox = Maildir/
smtpd_banner = $myhostname ESMTP
smtpd_relay_restrictions =
    permit_mynetworks,
    reject_unauth_destination
```

Enable the submission port in */etc/postfix/master.cf*:

```
sudo nano /etc/postfix/master.cf
# Uncomment (remove leading #) and ensure this block exists:
submission inet n - y - - smtpd
  -o syslog_name=postfix/submission
  -o smtpd_tls_security_level=may
  -o smtpd_sasl_auth_enable=yes
  -o smtpd_relay_restrictions=permit_sasl_authenticated,permit_mynetworks,reject
```

Without `localhost.localdomain` in `mydestination`, Postfix rejects locally generated mail from system daemons (e.g. cron output), causing bounce messages and failed deliveries. This addition is safe and required on Ubuntu.

Configure Dovecot

```
# 1. Protocols
sudo nano /etc/dovecot/dovecot.conf
# Verify or add:
protocols = imap

# 2. Mail location
sudo nano /etc/dovecot/conf.d/10-mail.conf
# Find and set:
mail_location = maildir:~/Maildir

# 3. Authentication
sudo nano /etc/dovecot/conf.d/10-auth.conf
# Set:
disable_plaintext_auth = no
auth_mechanisms = plain login

# 4. Postfix integration
sudo nano /etc/dovecot/conf.d/10-master.conf
```

```
# Inside the service auth { } block, add/uncomment:
unix_listener /var/spool/postfix/private/auth {
  mode = 0660
  user = postfix
  group = postfix
}
```

Create Mail Users

```
# Create one system account per team member with a Maildir:
for USER in abraham robine manda abba kana joyce; do
  sudo useradd -m -s /bin/bash "$USER"
  echo "$USER:Mail@123" | sudo chpasswd
  sudo -u "$USER" mkdir -p /home/$USER/Maildir/{new,cur,tmp}
  sudo chmod -R 700 /home/$USER/Maildir
  echo "Created mailbox for: $USER"
done

# Verify:
ls /home/
```

Start & Enable Services

```
sudo systemctl restart postfix dovecot
sudo systemctl enable postfix dovecot

# Check status:
sudo systemctl status postfix | grep Active
sudo systemctl status dovecot | grep Active

# Check listening ports:
ss -tlnp | grep -E '25|587|143'
# Expected:
#   *:25   -> postfix  (SMTP)
#   *:587  -> postfix  (submission)
#   *:143  -> dovecot  (IMAP)
```

Open Firewall Ports (if UFW active)

```
sudo ufw status
# If active:
sudo ufw allow 25/tcp
sudo ufw allow 587/tcp
sudo ufw allow 143/tcp
sudo ufw allow 22/tcp
sudo ufw reload
```

SMTP Verification & Testing

```
# From smtp-vm --- send test mail to a local user:
swaks --to abraham@enterprise.local \
      --from kana@enterprise.local \
      --server 192.168.100.15 \
      --port 25 \
      --body "Hello from the enterprise mail server!"

# Check delivery on smtp-vm:
sudo ls -la /home/abraham/Maildir/new/
# A new file should appear

# Read with mail command:
sudo -u abraham mail -f /home/abraham/Maildir

# From another VM (e.g. dns-vm or webserver-vm) --- install swaks first:
sudo apt install -y swaks mailutils
swaks --to robine@enterprise.local \
      --from abba@enterprise.local \
      --server 192.168.100.15 \
      --port 25 \
      --body "Cross-VM mail test"

# Test IMAP connectivity:
curl -v imap://192.168.100.15/ \
    --user robine:Mail@123 2>&1 | grep -E "< |> |Connected"

# Watch Postfix delivery log in real time:
sudo tail -f /var/log/mail.log
# Look for: status=delivered
```

ROBINE — DNS Server (dns-vm)

VM:	dns-vm
Role:	DNS Server — Bind9 Authoritative + Recursive
IP:	192.168.100.10

The DNS server resolves all `enterprise.local` hostnames to IPs and handles reverse lookups. It also forwards queries for external domains to Google (8.8.8.8) and Cloudflare (1.1.1.1).

Install Bind9

```
sudo apt update
sudo apt install -y bind9 bind9utils bind9-doc
sudo systemctl status bind9
```

Configure named.conf.options

Edit */etc/bind/named.conf.options*:

```
options {
  directory "/var/cache/bind";

  // Forward external queries to Google and Cloudflare:
  forwarders { 8.8.8.8; 1.1.1.1; };

  dnssec-validation auto;

  // Only listen on the LAN interface and loopback:
  listen-on { 192.168.100.10; 127.0.0.1; };

  // Accept queries from the entire enterprise subnet:
  allow-query { 192.168.100.0/24; 127.0.0.1; };

  recursion yes;
};
```

The original used `allow-query { any; };` which allows anyone on the internet to query your DNS server (open resolver). Restricting to `192.168.100.0/24` and `127.0.0.1` is both more secure and sufficient for all project VMs.

Configure Zones

Edit */etc/bind/named.conf.local*:

```
// Forward zone:
zone "enterprise.local" {
  type master;
  file "/etc/bind/db.enterprise.local";
};

// Reverse zone:
zone "100.168.192.in-addr.arpa" {
  type master;
  file "/etc/bind/db.192.168.100";
};
```

Forward Zone File

Create */etc/bind/db.enterprise.local*:

```
$TTL 604800
@  IN  SOA  dns.enterprise.local.  admin.enterprise.local. (
              2026051203  ; Serial (increment every time you edit this file)
              604800      ; Refresh (1 week)
              86400       ; Retry   (1 day)
              2419200     ; Expire  (4 weeks)
              604800 )    ; Negative Cache TTL

; Name servers:
@             IN  NS   dns.enterprise.local.

; A records:
@             IN  A    192.168.100.10
dns           IN  A    192.168.100.10
router        IN  A    192.168.100.1
files         IN  A    192.168.100.11
webserver     IN  A    192.168.100.14
www           IN  CNAME webserver
smtp          IN  A    192.168.100.15
mail          IN  CNAME smtp
monitor       IN  A    192.168.100.12
backup        IN  A    192.168.100.13
client1       IN  A    192.168.100.50
client2       IN  A    192.168.100.51

; MX record --- mail for enterprise.local goes to smtp-vm:
@             IN  MX   10  smtp.enterprise.local.
```

Reverse Zone File

Create */etc/bind/db.192.168.100*:

```
$TTL 604800
@  IN  SOA  dns.enterprise.local.  admin.enterprise.local. (
              2026051203  ; Serial
              604800      ; Refresh
              86400       ; Retry
              2419200     ; Expire
              604800 )    ; Negative Cache TTL

; Name servers:
@   IN  NS   dns.enterprise.local.
```

```
; PTR records (last octet -> hostname):
1   IN  PTR  router.enterprise.local.
10  IN  PTR  dns.enterprise.local.
11  IN  PTR  files.enterprise.local.
12  IN  PTR  monitor.enterprise.local.
13  IN  PTR  backup.enterprise.local.
14  IN  PTR  webserver.enterprise.local.
15  IN  PTR  smtp.enterprise.local.
50  IN  PTR  client1.enterprise.local.
51  IN  PTR  client2.enterprise.local.
```

Verify, Restart & Test

```
sudo named-checkconf
sudo named-checkzone enterprise.local /etc/bind/db.enterprise.local
sudo named-checkzone 100.168.192.in-addr.arpa /etc/bind/db.192.168.100

sudo systemctl restart bind9
sudo systemctl enable bind9

# Tests:
dig @192.168.100.10 router.enterprise.local    # -> 192.168.100.1
dig @192.168.100.10 webserver.enterprise.local # -> 192.168.100.14
dig @192.168.100.10 smtp.enterprise.local      # -> 192.168.100.15
dig @192.168.100.10 mail.enterprise.local      # -> CNAME smtp -> 192.168.100.15
dig @192.168.100.10 MX enterprise.local        # -> smtp.enterprise.local
dig @192.168.100.10 -x 192.168.100.15          # -> smtp.enterprise.local.
dig @192.168.100.10 google.com                 # External forwarding test
```

MANDA — File Server / Samba (fileserver-vm)

VM:	fileserver-vm
Role:	Samba File Server — Public + Private Shares
IP:	192.168.100.11

Install Samba

```
sudo apt update && sudo apt install -y samba
sudo systemctl status smbd | grep Active
```

Create Share Directories & Permissions

```
sudo mkdir -p /srv/samba/public /srv/samba/private
```

```
# Public share (world-accessible):
sudo chmod 777 /srv/samba/public
sudo chown nobody:nogroup /srv/samba/public

# Private share (staff group only):
sudo groupadd staff
sudo chown root:staff /srv/samba/private
sudo chmod 2770 /srv/samba/private

ls -la /srv/samba/
```

Create Samba User

```
sudo useradd -M -s /sbin/nologin staffuser
sudo smbpasswd -a staffuser       # Password: staff123
sudo usermod -aG staff staffuser
groups staffuser                  # Expected: staffuser : staffuser staff
```

Configure smb.conf

Replace the contents of */etc/samba/smb.conf*:

```
[global]
   workgroup = ENTERPRISE
   server string = Enterprise File Server %v
   security = user
   map to guest = Bad User
   guest account = nobody
   log file = /var/log/samba/log.%m
   max log size = 1000
   server role = standalone server
   passdb backend = tdbsam
   obey pam restrictions = yes
   unix password sync = yes

[public]
   comment = Public Share No Authentication Required
   path = /srv/samba/public
   browsable = yes
   writable = yes
   guest ok = yes
   read only = no
   create mask = 0666
   directory mask = 0777
   force user = nobody
```

```
[private]
   comment = Private Share Staff Only
   path = /srv/samba/private
   browsable = yes
   writable = yes
   guest ok = no
   valid users = @staff
   create mask = 0660
   directory mask = 0770
   force group = staff
```

Start Samba & Verify

```
testparm                                        # Should print: "Loaded services file OK."
sudo systemctl restart smbd nmbd
sudo systemctl enable smbd nmbd

# Tests:
smbclient -L localhost -U%                                  # List shares
smbclient //localhost/public -U% -c 'ls'
smbclient //localhost/private -U staffuser%staff123 -c 'ls'
echo "Test file" | sudo tee /srv/samba/public/test.txt

# Wrong password should give: NT_STATUS_LOGON_FAILURE
smbclient //localhost/private -U% -c 'ls'
```

ABBA — Web Server / Apache (webserver-vm)

VM:	webserver-vm
Role:	Apache Web Server + PHP Intranet Dashboard
IP:	192.168.100.14

Install Apache & PHP

```
sudo apt update
sudo apt install -y apache2 php libapache2-mod-php \
    php-curl php-gd php-xml php-mbstring

sudo a2enmod php8.1 rewrite
sudo systemctl start apache2
sudo systemctl enable apache2
sudo systemctl status apache2 | grep Active
```

On Ubuntu 22.04, the PHP module for Apache is `php8.1`, not `php`. Running `a2enmod php` will fail silently or enable the wrong module. Always specify the

version. If unsure, run `ls /etc/apache2/mods-available/ | grep php` to confirm.

Allow HTTP Through Firewall

```
sudo ufw status
# If active:
sudo ufw allow 80/tcp
sudo ufw allow 22/tcp
sudo ufw reload
```

Deploy the Intranet Status Dashboard

```
sudo rm -f /var/www/html/index.html
sudo tee /var/www/html/index.php << 'PHPEOF'
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="refresh" content="30">
  <title>Enterprise Network Dashboard</title>
  <style>
    * { box-sizing:border-box; margin:0; padding:0; }
    body { font-family:'Segoe UI',Arial,sans-serif;
           background:linear-gradient(135deg,#0D1B2A 0%,#1B4F72 100%);
           min-height:100vh; padding:30px; color:#ecf0f1; }
    .container { max-width:920px; margin:0 auto; }
    .header { text-align:center; padding:30px 0 20px; }
    .header h1 { font-size:2.2em; color:#AED6F1; letter-spacing:2px; }
    .header p  { color:#7FB3D3; margin-top:8px; font-size:0.95em; }
    .badge { display:inline-block; background:#2E86AB; color:white;
             padding:3px 10px; border-radius:12px; font-size:0.75em; }
    table { width:100%; border-collapse:collapse; margin-top:20px;
            background:rgba(255,255,255,0.05); border-radius:10px;
            overflow:hidden; }
    thead tr { background:#1B4F72; }
    th { padding:14px 18px; text-align:left; font-size:0.85em;
         text-transform:uppercase; letter-spacing:1px; color:#AED6F1; }
    td { padding:12px 18px; border-bottom:1px solid rgba(255,255,255,0.07); }
    .up   { color:#2ECC71; font-weight:bold; }
    .down { color:#E74C3C; font-weight:bold; }
    .ip   { font-family:monospace; color:#AED6F1; }
    .footer { text-align:center; padding:20px; color:#566573; font-size:0.8em; }
    .dot { display:inline-block; width:10px; height:10px;
           border-radius:50%; margin-right:6px; }
    .dot-up   { background:#2ECC71; box-shadow:0 0 6px #2ECC71; }
```

```
    .dot-down { background:#E74C3C; box-shadow:0 0 6px #E74C3C; }
  </style>
</head>
<body>
<div class="container">
  <div class="header">
    <h1>&#x1F5A7; Enterprise Network Dashboard</h1>
    <p>Domain: <span class="badge">enterprise.local</span>
         Network: <span class="badge">192.168.100.0/24</span></p>
  </div>
  <table>
    <thead>
      <tr><th>Service</th><th>Hostname</th><th>IP Address</th>
          <th>Status</th><th>Response</th></tr>
    </thead>
    <tbody>
<?php
$services = [
  ['Router/Gateway', 'router.enterprise.local',    '192.168.100.1',  'http'],
  ['DNS Server',     'dns.enterprise.local',       '192.168.100.10', 'http'],
  ['File Server',    'files.enterprise.local',     '192.168.100.11', 'http'],
  ['Web Server',     'webserver.enterprise.local', '192.168.100.14', 'http'],
  ['SMTP/Mail',      'smtp.enterprise.local',      '192.168.100.15', 'smtp'],
  ['Monitoring',     'monitor.enterprise.local',   '192.168.100.12', 'http'],
  ['Backup Server',  'backup.enterprise.local',    '192.168.100.13', 'http'],
];

foreach ($services as $svc) {
  [$name, $host, $ip, $type] = $svc;

  if ($type === 'smtp') {
    // Check SMTP: try TCP connect to port 25
    $fp  = @fsockopen($ip, 25, $en, $es, 2);
    $up  = ($fp !== false);
    if ($fp) fclose($fp);
    $resp = $up ? 'Port 25 open' : 'Port 25 closed';
  } else {
    // Check HTTP: try port 80
    $fp  = @fsockopen($ip, 80, $en, $es, 2);
    $up  = ($fp !== false);
    if ($fp) fclose($fp);
    $resp = $up ? 'Port 80 open' : 'Port 80 closed';
  }

  $dc = $up ? 'dot-up'   : 'dot-down';
  $sc = $up ? 'up'       : 'down';
```

```
  $st = $up ? 'UP'       : 'DOWN';
  echo "<tr><td><b>$name</b></td><td class='ip'>$host</td>"
     . "<td class='ip'>$ip</td>"
     . "<td><span class='dot $dc'></span><span class='$sc'>$st</span></td>"
     . "<td style='color:#7F8C8D;font-size:.85em'>$resp</td></tr>\n";
}
?>
    </tbody>
  </table>
  <div class="footer">
    Last updated: <?php echo date('Y-m-d H:i:s T'); ?>
    &bull; Auto-refreshes every 30 s &bull; Ubuntu 22.04 LTS
  </div>
</div>
</body>
</html>
PHPEOF

sudo chown www-data:www-data /var/www/html/index.php
sudo chmod 644 /var/www/html/index.php
sudo systemctl restart apache2
```

The original dashboard used `curl` (HTTP) for non-SMTP services, which gives misleading DOWN results for services like the router, file server, and DNS server that do not serve HTTP on port 80. The corrected version uses `fsockopen` with the relevant port for each service: port 80 for HTTP-capable hosts and port 25 for SMTP. This gives accurate UP/DOWN readings regardless of whether the service is a web server.

Verification

```
curl -I http://localhost                        # Expected: HTTP/1.1 200 OK
curl http://192.168.100.14/ | grep -i "Enterprise Network"
curl http://webserver.enterprise.local/         # After DNS is configured
sudo tail -20 /var/log/apache2/error.log        # Check for PHP errors
```

KANA — Monitoring / Zabbix (monitoring-vm)

VM:	monitoring-vm
Role:	Zabbix 6.4 Monitoring Server + Agents on All VMs
IP:	192.168.100.12

This is the most complex installation. Read each step fully before executing. Zabbix setup typically takes 20–30 minutes.

Install LAMP Stack

```
sudo apt update
sudo apt install -y apache2 mysql-server php php-mysql php-ldap \
    php-bcmath php-mbstring php-gd php-xml php-curl libapache2-mod-php

# Secure MySQL (follow prompts; set root password = zabbixroot):
sudo mysql_secure_installation
# Recommended answers:
#   Set root password?          -> Y, enter: zabbixroot
#   Remove anonymous users?     -> Y
#   Disallow root login remotely? -> Y
#   Remove test database?       -> Y
#   Reload privilege tables?    -> Y
```

Install Zabbix 6.4

```
wget https://repo.zabbix.com/zabbix/6.4/ubuntu/pool/main/z/zabbix-release/\
zabbix-release_6.4-1+ubuntu22.04_all.deb
sudo dpkg -i zabbix-release_6.4-1+ubuntu22.04_all.deb
sudo apt update
sudo apt install -y zabbix-server-mysql zabbix-frontend-php \
    zabbix-apache-conf zabbix-sql-scripts zabbix-agent
```

Create Zabbix Database & Import Schema

```
sudo mysql -uroot -pzabbixroot << 'SQL'
CREATE DATABASE zabbix CHARACTER SET utf8mb4 COLLATE utf8mb4_bin;
CREATE USER 'zabbix'@'localhost' IDENTIFIED BY 'zabbixpass';
GRANT ALL PRIVILEGES ON zabbix.* TO 'zabbix'@'localhost';
SET GLOBAL log_bin_trust_function_creators = 1;
FLUSH PRIVILEGES;
QUIT
SQL

# Import schema (takes 1-2 minutes -- wait until prompt returns):
zcat /usr/share/zabbix-sql-scripts/mysql/server.sql.gz \
  | mysql -uzabbix -pzabbixpass zabbix

# Disable trust setting after import:
sudo mysql -uroot -pzabbixroot \
  -e "SET GLOBAL log_bin_trust_function_creators = 0;"
```

Configure Zabbix Server

```
# Set database password:
sudo sed -i 's/^# DBPassword=.*/DBPassword=zabbixpass/' \
```

```
    /etc/zabbix/zabbix_server.conf
sudo sed -i 's/^DBPassword=.*/DBPassword=zabbixpass/' \
    /etc/zabbix/zabbix_server.conf
grep 'DBPassword' /etc/zabbix/zabbix_server.conf    # Verify

# Set PHP timezone:
sudo sed -i 's/^;date.timezone =.*/date.timezone = UTC/' \
    /etc/php/8.1/apache2/php.ini

sudo systemctl restart zabbix-server zabbix-agent apache2 mysql
sudo systemctl enable zabbix-server zabbix-agent apache2 mysql

# Check for errors:
sudo tail -20 /var/log/zabbix/zabbix_server.log
```

Complete Zabbix Web Setup (Browser)

Open http://192.168.100.12/zabbix from any browser on the internal network:

1. Click **Next step** on the welcome screen (all PHP checks should be green).
2. Database: Host=`localhost`, Port=`0`, Name=`zabbix`, User=`zabbix`, Password=`zabbixpass`.
3. Server name: `Enterprise Monitor`. Timezone: `UTC`.
4. Click **Next step** → **Finish**.
5. Default login: **Admin / zabbix** — change immediately after first login.

Add Hosts in Zabbix Web UI

After all VMs have installed the Zabbix agent (Section 10.7 below), add each host:

- Configuration → Hosts → **Create host**
- **Host name**: match the VM hostname exactly (e.g. `smtp-vm`).
- **Groups**: Linux servers.
- **Interfaces**: add Agent type, set IP to the VM's static IP.
- **Templates**: search and add *Linux by Zabbix agent*.
- Repeat for: router-vm, dns-vm, fileserver-vm, webserver-vm, smtp-vm, backup-vm, client1-vm, client2-vm.

For smtp-vm, add an additional TCP service check: Configuration → Hosts → smtp-vm → Items → Create item → key: `net.tcp.port[,25]`, type: *Zabbix agent*.

Install Zabbix Agent on Every Other VM

This sub-section must be executed by **every team member** on their own VM — including ABRAHAM on smtp-vm.

```
sudo apt update
sudo apt install -y zabbix-agent

# Configure agent to report to monitoring-vm:
sudo sed -i 's/^Server=127.0.0.1/Server=192.168.100.12/' \
    /etc/zabbix/zabbix_agentd.conf
sudo sed -i 's/^ServerActive=127.0.0.1/ServerActive=192.168.100.12/' \
    /etc/zabbix/zabbix_agentd.conf
sudo sed -i "s/^Hostname=Zabbix server/Hostname=$(hostname)/" \
    /etc/zabbix/zabbix_agentd.conf

# Verify:
grep -E '^(Server|ServerActive|Hostname)' /etc/zabbix/zabbix_agentd.conf

sudo systemctl restart zabbix-agent
sudo systemctl enable zabbix-agent
sudo systemctl status zabbix-agent | grep Active
```

JOYCE — Backup Server (backup-vm)

VM:	backup-vm + client1-vm + client2-vm
Role:	rsync Backup Server + Client VMs
IP:	192.168.100.13 (backup-vm)

Generate SSH Key & Distribute

```
# On backup-vm:
ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub
# Copy the entire output line (starts with ssh-rsa ...)

# On EACH remote VM (run as sysadmin on each target):
mkdir -p ~/.ssh && chmod 700 ~/.ssh
echo "PASTE_PUBLIC_KEY_HERE" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# Back on backup-vm, test passwordless SSH (each must return hostname without a password pr
ssh -o StrictHostKeyChecking=no sysadmin@192.168.100.1  "hostname"  # router-vm
ssh -o StrictHostKeyChecking=no sysadmin@192.168.100.10 "hostname"  # dns-vm
ssh -o StrictHostKeyChecking=no sysadmin@192.168.100.11 "hostname"  # fileserver-vm
```

```
ssh -o StrictHostKeyChecking=no sysadmin@192.168.100.14 "hostname"  # webserver-vm
ssh -o StrictHostKeyChecking=no sysadmin@192.168.100.15 "hostname"  # smtp-vm
ssh -o StrictHostKeyChecking=no sysadmin@192.168.100.12 "hostname"  # monitoring-vm
```

Create Backup Directories

```
sudo mkdir -p /backup/{router,dns,fileserver,webserver,smtp,monitoring}
sudo chown sysadmin:sysadmin /backup -R
ls -la /backup/
```

Create the Backup Script

```
tee /home/sysadmin/backup.sh << 'EOF'
#!/bin/bash
# Enterprise Network Backup Script -- runs nightly at 02:00 via cron

DATE=$(date +%Y%m%d_%H%M)
BACKUP_ROOT="/backup"
LOG_PREFIX="[$(date '+%Y-%m-%d %H:%M:%S')]"

echo "$LOG_PREFIX ===== Backup started ====="

# --- Router: DHCP config and iptables rules ---
echo "$LOG_PREFIX Backing up router-vm..."
rsync -avz --delete sysadmin@192.168.100.1:/etc/dhcp/ \
    $BACKUP_ROOT/router/dhcp_$DATE/ 2>/dev/null
rsync -avz --delete sysadmin@192.168.100.1:/etc/iptables/ \
    $BACKUP_ROOT/router/iptables_$DATE/ 2>/dev/null
rsync -avz sysadmin@192.168.100.1:/etc/sysctl.conf \
    $BACKUP_ROOT/router/sysctl_$DATE.conf 2>/dev/null

# --- DNS: Bind9 zone files and config ---
echo "$LOG_PREFIX Backing up dns-vm..."
rsync -avz --delete sysadmin@192.168.100.10:/etc/bind/ \
    $BACKUP_ROOT/dns/bind_$DATE/

# --- File Server: Samba config and private share data ---
echo "$LOG_PREFIX Backing up fileserver-vm..."
rsync -avz --delete sysadmin@192.168.100.11:/etc/samba/ \
    $BACKUP_ROOT/fileserver/samba_$DATE/
rsync -avz --delete sysadmin@192.168.100.11:/srv/samba/private/ \
    $BACKUP_ROOT/fileserver/private_$DATE/

# --- Web Server: Apache config and web content ---
echo "$LOG_PREFIX Backing up webserver-vm..."
rsync -avz --delete sysadmin@192.168.100.14:/var/www/html/ \
```

```
    $BACKUP_ROOT/webserver/html_$DATE/
rsync -avz --delete sysadmin@192.168.100.14:/etc/apache2/ \
    $BACKUP_ROOT/webserver/apache_$DATE/

# --- SMTP Server: Postfix and Dovecot configs ---
echo "$LOG_PREFIX Backing up smtp-vm..."
rsync -avz --delete sysadmin@192.168.100.15:/etc/postfix/ \
    $BACKUP_ROOT/smtp/postfix_$DATE/ \
    && echo "$LOG_PREFIX Postfix config: OK"
rsync -avz --delete sysadmin@192.168.100.15:/etc/dovecot/ \
    $BACKUP_ROOT/smtp/dovecot_$DATE/ \
    && echo "$LOG_PREFIX Dovecot config: OK"

# --- Monitoring: Zabbix config ---
echo "$LOG_PREFIX Backing up monitoring-vm..."
rsync -avz --delete sysadmin@192.168.100.12:/etc/zabbix/ \
    $BACKUP_ROOT/monitoring/zabbix_$DATE/

# --- Cleanup: remove backups older than 7 days ---
echo "$LOG_PREFIX Cleaning up old backups (>7 days)..."
find $BACKUP_ROOT -maxdepth 2 -type d -name "*_[0-9]*" \
    -mtime +7 -exec rm -rf {} \; 2>/dev/null

echo "$LOG_PREFIX ===== Backup completed ====="
echo "$LOG_PREFIX Disk usage: $(du -sh $BACKUP_ROOT)"
EOF

chmod +x /home/sysadmin/backup.sh
```

Schedule with Cron

```
crontab -e
# Add this line (runs backup every night at 02:00):
0 2 * * * /home/sysadmin/backup.sh >> /home/sysadmin/backup.log 2>&1

crontab -l | grep backup   # Verify the line was saved

# Test manually before relying on cron:
/home/sysadmin/backup.sh
cat /home/sysadmin/backup.log
ls -la /backup/smtp/          # Verify SMTP backup created
```

Client VMs — client1-vm and client2-vm

Apply the Netplan from Section 4.3.3, then install tools and test:

```
sudo apt update
sudo apt install -y smbclient curl dnsutils swaks mailutils

# Connectivity tests:
ip addr show enp0s8 | grep inet      # Should be in 192.168.100.50-200 range
ip route | grep default              # via 192.168.100.1
resolvectl status | grep 'DNS Servers'  # 192.168.100.10
ping -c 3 192.168.100.1              # Gateway
ping -c 3 192.168.100.15             # smtp-vm
dig @192.168.100.10 smtp.enterprise.local  # Should resolve to .15

# Send test mail from client:
swaks --to abraham@enterprise.local \
      --from client1@enterprise.local \
      --server 192.168.100.15 --port 25 \
      --body "Test from client1-vm"
```

Integration Testing — Full Validation Suite

Once all VMs are configured and running, perform these numbered tests in order. Document every output (screenshots + copy-pasted terminal) for the report.

Start VMs in this order: **router-vm first**, then **dns-vm**, then all others. Allow 30 seconds after each VM starts before proceeding.

Test 1 — Network Connectivity

```
echo "=== Test 1: Network Connectivity ==="
for IP in 192.168.100.1 192.168.100.10 192.168.100.11 \
          192.168.100.14 192.168.100.15 192.168.100.12 \
          192.168.100.13 8.8.8.8; do
  ping -c 1 -W 2 $IP &>/dev/null \
    && echo "PASS: $IP reachable" \
    || echo "FAIL: $IP unreachable"
done
```

Test 2 — DNS Resolution

```
echo "=== Test 2: DNS Resolution ==="
for HOST in router dns files webserver smtp mail monitor backup; do
  echo -n "$HOST.enterprise.local -> "
  dig @192.168.100.10 $HOST.enterprise.local +short
done
echo "--- MX record ---"
dig @192.168.100.10 MX enterprise.local +short
```

```
echo "--- Reverse .15 ---"
dig @192.168.100.10 -x 192.168.100.15 +short   # -> smtp.enterprise.local.
echo "--- External ---"
dig @192.168.100.10 google.com +short | head -2
```

Test 3 — DHCP Lease (run from client VMs)

```
echo "=== Test 3: DHCP Lease ==="
ip addr show enp0s8 | grep "inet "       # 192.168.100.50-200
ip route | grep default                  # via 192.168.100.1
resolvectl status | grep 'DNS Servers'   # 192.168.100.10
```

Test 4 — Web Server

```
echo "=== Test 4: Web Server ==="
curl -s -o /dev/null -w "%{http_code}" http://192.168.100.14/
# Expected: 200
curl http://webserver.enterprise.local/ | grep -i "Enterprise Network Dashboard"
```

Test 5 — SMTP Mail Server

```
echo "=== Test 5: SMTP Mail Server ==="

# 5a. Port checks on smtp-vm:
nc -zv 192.168.100.15 25     # SMTP
nc -zv 192.168.100.15 587    # Submission
nc -zv 192.168.100.15 143    # IMAP
# Expected: Connection to 192.168.100.15 ... succeeded!

# 5b. Send test mail via swaks:
swaks --to robine@enterprise.local \
      --from abba@enterprise.local \
      --server 192.168.100.15 --port 25 \
      --body "Integration test mail"
# Expected: <- 250 2.0.0 Ok: queued as ...

# 5c. Verify delivery on smtp-vm:
ssh sysadmin@192.168.100.15 "sudo ls /home/robine/Maildir/new/"
# Expected: at least one file listed

# 5d. IMAP connectivity:
curl -v imap://192.168.100.15/ \
    --user robine:Mail@123 2>&1 | grep -E "< |Connected"

# 5e. View mail log:
```

```
ssh sysadmin@192.168.100.15 "sudo tail -10 /var/log/mail.log"
# Expected: status=delivered
```

Test 6 — Samba File Server

```
echo "=== Test 6: Samba File Server ==="
smbclient -L //192.168.100.11 -U%                          # List shares
smbclient //192.168.100.11/public -U% -c 'ls'
smbclient //192.168.100.11/private \
    -U staffuser%staff123 -c 'ls'
# Wrong credentials should give NT_STATUS_LOGON_FAILURE:
smbclient //192.168.100.11/private -U% -c 'ls'
```

Test 7 — Zabbix Monitoring

```
echo "=== Test 7: Zabbix Monitoring ==="
sudo systemctl status zabbix-server | grep Active
curl -s -o /dev/null -w "%{http_code}" http://192.168.100.12/zabbix
# Expected: 200
# Verify all hosts (including smtp-vm) show green ZBX icon in Web UI
nc -zv 192.168.100.15 10050    # Zabbix agent port on smtp-vm
```

Test 8 — Backup Verification

```
echo "=== Test 8: Backup Verification ==="
/home/sysadmin/backup.sh
cat /home/sysadmin/backup.log | tail -30
ls -la /backup/smtp/
ls /backup/smtp/postfix_*/main.cf    # Confirm Postfix config was backed up
```

Test 9 — Full End-to-End from Client

```
echo "=== Test 9: Full End-to-End from client1-vm ==="
echo "1. DHCP IP:     $(ip addr show enp0s8 | grep 'inet ' | awk '{print $2}')"
echo "2. Gateway:     $(ip route | grep default | awk '{print $3}')"
echo "3. DNS forward: $(dig @192.168.100.10 webserver.enterprise.local +short)"
echo "4. DNS smtp:    $(dig @192.168.100.10 smtp.enterprise.local +short)"
echo "5. DNS reverse: $(dig @192.168.100.10 -x 192.168.100.15 +short)"
echo "6. Internet:    $(ping -c1 8.8.8.8 -q 2>&1 | grep received)"
echo "7. Web server:  $(curl -s -o /dev/null -w 'HTTP %{http_code}' http://192.168.100.14)"
echo "8. SMTP p25:    $(nc -zv 192.168.100.15 25 2>&1 | grep -o 'succeeded\|failed')"
echo "9. Samba pub:   $(smbclient //192.168.100.11/public -U% -c 'ls' 2>&1 | grep -c blocks)
echo "=== All tests complete ==="
```

Troubleshooting Reference

Problem	Likely Cause
VMs only talk on same host	Adapter 1 was NAT instead of Bridg
No internet on any VM	Router NAT off or IP forwarding dis
VM cannot ping router	Wrong netplan or adapter not on int
DHCP lease not assigned	isc-dhcp-server not running or wrong
DNS fails (NXDOMAIN)	Bind9 not running or zone syntax er
SMTP: Connection refused on port 25	Postfix not running or bound to wro
Mail not delivered (status=bounced)	User does not exist or mydestination
IMAP: AUTH failed	Dovecot plaintext auth disabled
Maildir not found	Maildir not created for user
Samba: NT_STATUS_LOGON_FAILURE on private share	staffuser not in staff group or wrong
Apache 403 or connection refused	Apache not running or wrong permis
PHP module not loading	Wrong module name for Ubuntu 22.(
Zabbix agent not reporting	Agent misconfigured or not running
Backup: Permission denied	SSH key not distributed to target VI
netplan apply: file permissions warning	File is world-readable
enp0s3 has 10.0.2.x address	Adapter 1 is NAT instead of Bridged

Deliverables Checklist

Deliverable	Owner	Description
Network Diagram	ABRAHAM	All VMs, IPs, interconnections, services, adapter types (Bridg
Config Files Archive	All	Zip: netplan YAMLs, dhcpd.conf, named.conf.*, zone files, sn
Screenshots (numbered)	All	VM desktop showing hostname+IP, each service `systemctl`
Test Logs	All	Copy-paste terminal output for all 9 tests in Section 12.
SMTP Test Evidence	ABRAHAM	*/var/log/mail.log* excerpt showing mail delivered; swaks ou
Zabbix Dashboard Screenshot	KANA	Browser screenshot showing all VMs (including smtp-vm) wit
Web Dashboard Screenshot	ABBA	Browser screenshot of http://192.168.100.14/ showing SMTP
Backup Log	JOYCE	*/home/sysadmin/backup.log* after successful run; `ls /backu`
Team Contribution Table	Team Leader	Table with each person's name, VM(s), and specific tasks con
Demo Video (2–3 min)	All	Screen recording: DHCP lease, DNS (incl. smtp), mail send/r

Appendix — Quick Reference Commands

Service Status Checks

```
sudo systemctl status isc-dhcp-server     # Router
sudo systemctl status bind9               # DNS
sudo systemctl status smbd nmbd           # File Server
sudo systemctl status apache2             # Web Server
sudo systemctl status postfix dovecot     # SMTP (smtp-vm)
sudo systemctl status zabbix-server       # Monitoring
sudo systemctl status zabbix-agent        # All VMs
```

Network Diagnostics

```
ip addr show                                        # All IPs
ip route                                            # Routing table
ss -tlnp                                            # Open ports
dig @192.168.100.10 smtp.enterprise.local           # DNS test
sudo dhclient -r enp0s8 && sudo dhclient enp0s8 # Renew DHCP
cat /var/lib/dhcp/dhcpd.leases                      # DHCP leases (router only)
sudo iptables -L -v -n                              # Firewall rules
traceroute 192.168.100.1                            # Trace route to gateway

# Check that router's bridged interface has a real LAN IP (NOT 10.0.2.x):
ip addr show enp0s3    # Should show 192.168.x.x or 10.x.x.x from your LAN DHCP
```

SMTP Quick Reference

```
# View mail queue:
mailq

# Flush mail queue:
sudo postfix flush

# Check Postfix config:
postconf -n | grep -E 'myhostname|mydomain|mydestination|mynetworks'

# Send test mail (from any VM with swaks installed):
swaks --to USER@enterprise.local --server 192.168.100.15 --port 25

# View mail log:
sudo tail -f /var/log/mail.log

# Check IMAP with curl:
curl -v imap://192.168.100.15/ --user USER:Mail@123

# Restart both mail services:
sudo systemctl restart postfix dovecot
```

Service Restart Quick Reference

```
sudo systemctl restart isc-dhcp-server                          # Router
sudo systemctl restart bind9                                    # DNS
sudo systemctl restart smbd nmbd                                # File Server
sudo systemctl restart apache2                                  # Web Server
sudo systemctl restart postfix dovecot                          # SMTP
sudo systemctl restart zabbix-server zabbix-agent apache2 mysql # Monitoring
sudo systemctl restart zabbix-agent                             # All other VMs
```

Backup Manual Run & Check

```
/home/sysadmin/backup.sh
tail -30 /home/sysadmin/backup.log
ls -lR /backup/ | grep "^/backup"
ls /backup/smtp/      # Verify SMTP backup present
df -h /backup         # Check disk space
```

Summary of All Fixes Applied in This Document

#	Original	Fix Applied
1	Adapter 1: NAT on router-vm	Changed to **Bridged Adapter** — root cause
2	Non-router VMs had `enp0s3: dhcp4: true` in netplan	Removed; these VMs have no Adapter 1; only
3	Router netplan set a default route on `enp0s8`	Removed; default route must come from bridg
4	`127.0.0.1 YOUR_HOSTNAME` in */etc/hosts*	Changed to `127.0.1.1` — Ubuntu standard fo
5	`allow-query { any; };` in Bind9	Restricted to `192.168.100.0/24` and `127.0.0`
6	`mydestination` missing `localhost.localdomain`	Added to prevent bounce errors from local dae
7	Dashboard used curl/HTTP to check all services	Changed to `fsockopen` on correct port per ser
8	`a2enmod php` (wrong on Ubuntu 22.04)	Changed to `a2enmod php8.1`

This document covers every command, configuration file, and verification step needed
to build a fully functional enterprise network. Follow it in order, verify each step
before proceeding, and document your output for the report. Good luck!
